

Prueba Técnica

Procesamiento de Eventos en Tiempo Real MongoDB, Airflow, FastAPI y Docker

Quipux

24 de junio de 2026

1. Objetivo

Diseñar e implementar una plataforma capaz de consumir eventos desde una API pública, procesarlos en tiempo real (o near real-time), almacenarlos en MongoDB y generar reportes periódicos utilizando Airflow, FastAPI y Pydantic.

La solución debe seguir principios de diseño desacoplado, buenas prácticas de desarrollo y ejecutarse completamente mediante Docker Compose.

2. Contexto

El sistema debe monitorear eventos sísmicos publicados por el servicio USGS Earthquake Program.

API pública:

https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/all_hour.geojson

La API devuelve los terremotos detectados durante la última hora.

Ejemplo simplificado de respuesta:

```
{
  "features": [
    {
      "id": "us7000xxxx",
      "properties": {
        "mag": 4.2,
        "place": "20 km NW of California",
        "time": 1718610000000
      },
      "geometry": {
        "coordinates": [-120.12, 35.44, 10.5]
      }
    }
  ]
}
```

3. Requerimientos Funcionales

3.1. 1. Servicio de Ingesta

Desarrollar un servicio encargado de consultar la API cada 3 minutos.
El servicio debe:

- Consumir la API pública.
- Detectar nuevos eventos.
- Evitar registros duplicados.
- Transformar la respuesta a un modelo interno.
- Registrar errores y eventos relevantes mediante logs.

Modelo sugerido:

```
{  
  "event_id": "us7000xxxx",  
  "magnitude": 4.2,  
  "location": "20 km NW of California",  
  "latitude": 35.44,  
  "longitude": -120.12,  
  "depth": 10.5,  
  "event_time": "2026-06-17T10:30:00Z"  
}
```

3.2. 2. Procesamiento en Tiempo Real

Cada nuevo evento debe ser procesado inmediatamente después de ser recibido.
Calcular y actualizar métricas como:

- Cantidad de sismos por hora.
- Magnitud promedio.
- Magnitud máxima registrada.
- Distribución por rangos de magnitud (definidos por el candidato).

3.3. 3. Persistencia

MongoDB es la base de datos recomendada. Se aceptan otras bases de datos NoSQL siempre que la elección esté debidamente justificada.

Además de almacenar la información, se espera que el candidato:

- Diseñe adecuadamente las colecciones.
- Defina índices para optimizar consultas frecuentes.
- Justifique las decisiones de modelado de datos.

Colecciones sugeridas:

3.3.1. earthquakes

```
{
  "_id": "...",
  "event_id": "us7000xxxx",
  "magnitude": 4.2,
  "location": "...",
  "event_time": "..."
}
```

3.3.2. metrics

```
{
  "_id": "...",
  "window": "2026-06-17T10",
  "earthquake_count": 25,
  "avg_magnitude": 3.8,
  "max_magnitude": 6.1
}
```

4. API REST

Implementar una API utilizando FastAPI.

Como mínimo, se esperan los siguientes endpoints:

GET /earthquakes

GET /metrics

GET /reports

Se valorará positivamente la implementación de:

- Filtros.
- Paginación.
- Ordenamiento.
- Validación de parámetros mediante Pydantic.

5. Airflow

Implementar un DAG que se ejecute cada hora.

El flujo debe:

1. Leer los eventos almacenados.
2. Generar un reporte consolidado.
3. Persistir el resultado.

Colección sugerida:

hourly_reports

Ejemplo:

```
{
  "report_date": "2026-06-17T10:00:00Z",
  "total_events": 120,
  "average_magnitude": 3.9,
  "max_magnitude": 6.8,
  "top_locations": [
    "California",
    "Alaska",
    "Chile"
  ]
}
```

Consideraciones:

- Debe registrar eventos relevantes para facilitar el monitoreo.

6. Arquitectura

La siguiente estructura es únicamente una referencia. El candidato puede proponer una organización diferente siempre que mantenga una clara separación de responsabilidades.

```
src/
|-- deploy/
|   |-- Dockerfile
|
|-- config/
|   |-- settings.py
|   |-- logging.py
|   |-- constants.py
|
|-- metadata/
|   |-- schemas.py
|   |-- validators.py
|
|-- models/
|   |-- earthquake.py
|   |-- metric.py
|   |-- report.py
|
|-- database/
|   |-- mongodb.py
|   |-- indexes.py
|
|-- clients/
|   |-- usgs_client.py
|
|-- services/
|   |-- ingestion_service.py
|   |-- processing_service.py
|   |-- metrics_service.py
|   |-- reporting_service.py
```

```
|  
|-- api/  
|   |-- routes/  
|   \-- main.py  
|  
|-- airflow/  
|   \-- dags/  
|  
|-- utils/  
|  
|-- requirements.txt  
|-- docker-compose.yml  
\-- main.py
```

Se evaluará:

- Separación de responsabilidades.
- Modularización.
- Mantenibilidad.
- Escalabilidad.
- Aplicación de principios SOLID.

7. Docker

Toda la solución debe ejecutarse mediante uno o varios contenedores Docker.
La aplicación debe poder iniciarse utilizando:

```
docker compose up
```

Servicios mínimos esperados:

- MongoDB.
- API REST.
- Servicio de ingesta.

Consideraciones:

- Utilizar variables de entorno para configuración.
- No incluir credenciales hardcodeadas.
- Documentar el proceso de despliegue.

8. Bonificaciones

8.1. Nivel 1

- Pydantic.
- Logging estructurado.

- Modularización.
- Principios SOLID.
- Gestión eficiente de conexiones a base de datos.
- Estrategias de caché.

8.2. Nivel 2

Agregar observabilidad:

- Prometheus.
- Grafana.
- Métricas personalizadas de la aplicación.

8.3. Nivel 3

Implementar alguna de las siguientes alternativas:

- Kafka.
- RabbitMQ.
- WebSockets para actualización en tiempo real.
- Procesamiento orientado a eventos.

8.4. Nivel 4

Diseñar una arquitectura de datos orientada a analítica avanzada y Machine Learning.
La propuesta debe contemplar:

- Dashboards históricos para análisis de tendencias.
- Dashboards en tiempo real con actualización continua de eventos sísmicos.
- Generación de datasets analíticos para entrenamiento de modelos de Machine Learning.
- Procesamiento de archivos Parquet en tiempo real o near real-time.
- Separación entre capas transaccionales y analíticas.
- Estrategia de almacenamiento para datos históricos de gran volumen.

9. Supuestos

No es necesario implementar:

- Autenticación.
- Autorización.
- Frontend.
- Despliegues en proveedores cloud.
- CI/CD.

10. Entregables

- Repositorio Git.
- README con instrucciones de ejecución.
- Docker Compose funcional.
- Código fuente.
- Colección Postman.
- Diagrama de arquitectura.